Implementación de un Ecosistema NAS y Servidor de Aplicaciones en Hardware antiguo (Acer Aspire One)

Vargas Garcia Brandon Fabián
E-mail: brandondinog99@gmail.com

***Resumen*—Este reporte documenta la optimización extrema y despliegue de servicios NAS y de entretenimiento en una notebook Acer Aspire One (Intel Atom, 2 GB RAM). Se analiza la arquitectura de software diseñada para minimizar el impacto en el procesador mediante el renderizado WebGL en cliente (Three.js), la persistencia de servicios mediante systemd y la exposición segura a redes externas a través de configuraciones de Port Forwarding y Proxy Inverso bajo protocolos TLS 1.3.**

***Index Terms*—Intel Atom, Port Forwarding, Persistence, Flask, Three.js, NAS, UDP, Seguridad.**

I. Introducción

La reutilización de hardware considerado obsoleto representa un desafío de optimización. En este proyecto, se transformó una Acer Aspire One con procesador Intel Atom en un servidor NAS y de HaloCE funcional. La restricción de 2 GB de RAM dictó una arquitectura basada en servicios *headless* (sin interfaz gráfica), maximizando la capacidad del CPU para el procesamiento de peticiones y el manejo de tráfico híbrido TCP/UDP.

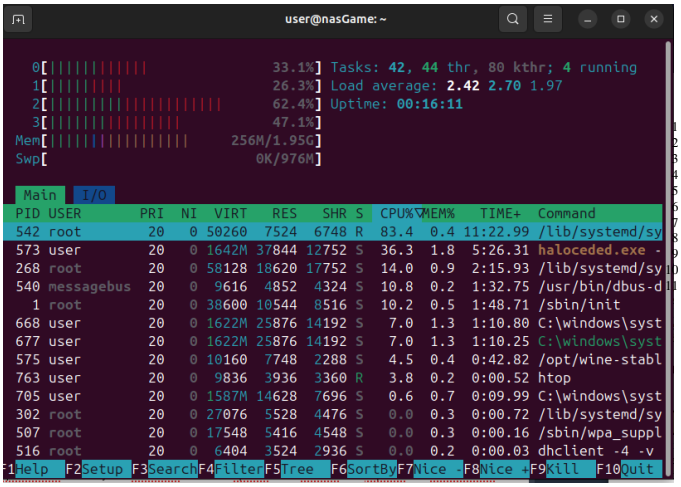

Figura 1: Demostracion de uso de recursos del server.

II. Desarrollo

II-1. Justificación de Debian frente a otras distribuciones: La elección de **Debian GNU/Linux** sobre alternativas como Ubuntu o Arch Linux se basó en los siguientes criteriose:

- **Soporte de Arquitectura Legacy:** A diferencia de las versiones modernas de Ubuntu Server que han discontinuado el soporte para *x86_32*, Debian mantiene la rama *i386* como arquitectura oficial, permitiendo el aprovechamiento total de la instrucción nativa del procesador Intel Atom.
- **Ausencia de Bloatware:** Mediante una instalación mínima (*netinst*), se evitaron servicios innecesarios como *snapd* o *unattended-upgrades*, los cuales generan picos de uso de CPU que podrían causar *lag* en el servidor de Halo CE.
- **Ecosistema de Paquetes Apt:** El gestor de paquetes `apt` y los repositorios estables garantizan que las dependencias de *Wine* para el servidor de videojuegos y *Bcrypt* para la seguridad se mantengan consistentes, minimizando el riesgo de fallos por incompatibilidad de librerías (*dependency hell*).

II-A. Persistencia de Servicios: Systemd

Para garantizar que el NAS y el servidor de Halo se inicien automáticamente en cada arranque o tras un fallo eléctrico, se crearon unidades de servicio en *systemd*. Esto asegura que el sistema sea autónomo y no requiera intervención manual vía SSH tras un reinicio.

```
[Unit]
Description=NAS Chapingo
After=network.target nginx.service

[Service]
ExecStart=/usr/bin/bash /home/NAS/script.sh
Restart=always
User=user

[Install]
WantedBy=multi-user.target
```

Listing 1: Ejemplo de configuracion del servicio para el NAS

II-B. Backend

El servidor utiliza Flask y SQLite. Una decisión de arquitectura crítica fue delegar el renderizado 3D de la malla dinámica a **Three.js** en el lado del cliente. Al descargar el procesamiento gráfico de la CPU de la Acer Aspire One hacia el navegador del usuario, se mantuvo la estabilidad del sistema.

Para la persistencia de información sensible, se utilizó **SQLite**, aprovechando su integración nativa en Debian Server. Esta base de datos gestiona el control de acceso basado en roles (RBAC), almacenando identificadores, privilegios y credenciales.

Con el objetivo de mitigar riesgos en caso de que la base de datos llegase a ser extraida, las contraseñas no se almacenan en texto plano. Se implementó la librería *Bcrypt* en el backend para el proceso de *hashing*. Este algoritmo incorpora un *salt*(una sal) aleatorio y un factor de costo ajustable, lo que vuelve computacionalmente inviable un ataque de fuerza bruta o el uso de tablas de arcoíris (*rainbow tables*).

II-C. Despliegue de Producción con Gunicorn y Systemd

En la fase de producción, el servidor de desarrollo de Flask resulta insuficiente por su naturaleza monohilo. Por ello, se implementó **Gunicorn** (*Green Unicorn*), un servidor WSGI que permite manejar múltiples peticiones concurrentes mediante procesos *workers*.

Para automatizar la ejecución y asegurar que el servicio inicie tras un reinicio del hardware, se creó una unidad de sistema en `systemd` (`webapp.service`). Este archivo de configuración define el entorno de ejecución y el enlace al puerto local:

```
[Unit]
Description=Aplicacion flask con gunicorn
After=network.target

[Service]
User=user
Group=www-data
WorkingDirectory=/srv/app
ExecStart=/srv/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:5000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

Listing 2: Archivo de configuración webapp.service

El ciclo de vida del servicio se gestiona mediante comandos de sistema que aseguran la limpieza de procesos previos y la persistencia:

```
sudo systemctl daemon-reload
sudo fuser -k 5000/tcp # Limpieza de puertos colgados
sudo systemctl start webapp
sudo systemctl enable webapp # Inicio automtico
```

II-D. Port Forwarding en el Router

Para permitir el acceso desde internet, se configuró el enrutamiento de puertos en el gateway perimetral. Esta técnica asocia la IP pública del hogar con la IP privada estática del servidor:

- **TCP 443:** Redirigido al puerto interno 443 del NAS para el tráfico web cifrado.
- **UDP 2302:** Redirigido para el tráfico de baja latencia del servidor de Halo.

La seguridad perimetral del servidor se gestionó mediante **UFW (Uncomplicated Firewall)**. Se aplicó una política de "denegación por defecto", exponiendo únicamente los servicios críticos para el funcionamiento del ecosistema y el servidor de Halo: Combat Evolved.

Para la administración del sistema, se restringió el acceso vía **SSH** únicamente a la red local, garantizando que la gestión de la máquina solo sea posible desde la estación de trabajo del administrador. Los comandos ejecutados para establecer las reglas de filtrado fueron:

```
# Apertura de puertos para trafico Web y Halo CE
sudo ufw allow 80/tcp
sudo ufw allow 2302/udp
sudo ufw allow 2303/udp

# Verificacion de reglas activas
sudo ufw show added
```

Listing 3: Configuracion de reglas de filtrado en UFW

Al listar las reglas activas, se confirmó la correcta jerarquía de permisos, asegurando la disponibilidad de los puertos necesarios mientras se mantiene el resto del espectro de red cerrado:

- **22/tcp (SSH):** Limitado a administración local.
- **80/tcp:** Tráfico HTTP para validación y redirección.
- **2302/udp y 2303/udp:** Puertos dedicados a la comunicación de baja latencia del servidor de Halo.

Figura 2: Configuración del mapeo de puertos en la interfaz del router.

II-E. Proxy Inverso y Seguridad TLS

NGINX actúa como la cara pública del servidor, gestionando el handshake SSL. Gracias a esto, la aplicación Flask en el puerto 5000 queda protegida tras una capa de cifrado administrada por Let's Encrypt.

II-E1. Seguridad mediante Proxy Inverso: La arquitectura utiliza **Nginx** como un *Proxy Inverso*. Esta configuración es vital para la seguridad por dos razones:

1. **Aislamiento:** El servidor Flask no se expone directamente a Internet, sino que escucha solo en `127.0.0.1`. Nginx actúa como un túnel seguro que filtra las peticiones.
2. **Terminación SSL:** Nginx gestiona el cifrado pesado, liberando de esta carga al backend y permitiendo una gestión centralizada de certificados.

La configuración en `/etc/nginx/sites-available/default` garantiza la redirección forzosa de tráfico inseguro (puerto 80) hacia el túnel cifrado (puerto 443):

```
server {
    listen 80;
    server_name server0069.duckdns.org;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name server0069.duckdns.org;

    # Certificados Let's Encrypt
    ssl_certificate /etc/letsencrypt/live/server0069.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/server0069.duckdns.org/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Listing 4: Configuración del Proxy Inverso en Nginx

```
nginx.service - High performance web
server...
Active: active (running) since Sat
2026-03-14...
```

Figura 3: Estado del servicio Nginx operando correctamente bajo carga.

II-E2. Configuración de Dominio y DNS Dinámico: Para permitir el acceso mediante un nombre de host legible y evitar la pérdida de conectividad por cambios en la dirección IP pública del ISP, se utilizó el servicio de DNS dinámico **DuckDNS**. Se configuró una **IP estática** en el enrutador para el servidor, garantizando que el mapeo de puertos y la actualización del registro en DuckDNS se mantengan constantes.

La seguridad del dominio se garantizó mediante la certificación SSL/TLS de **Let's Encrypt**. La integración se realizó con *Certbot*, permitiendo la renovación automática y el cifrado de extremo a extremo:

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx
```

Listing 5: Instalación de Certbot y despliegue de certificados

II-F. Mitigación de Ataques y Seguridad en la Capa de Aplicación

Para robustecer la resistencia del servidor frente a intentos de intrusión y abusos de servicio, se implementaron múltiples capas de defensa activa y pasiva:

- **Fail2Ban (Prevención de Fuerza Bruta):** Se configuró *Fail2Ban* para monitorear los registros de acceso. Este servicio identifica patrones de intentos fallidos de inicio de sesión y bloquea automáticamente las direcciones IP de origen mediante reglas dinámicas en el firewall, mitigando ataques de fuerza bruta dirigidos al SSH y al panel del NAS.
- **Gestión de Archivos y Listas Blancas:** El servidor implementa una política de "denegación por defecto" en la carga de archivos. Solo se permiten extensiones específicas (ej. `.txt, .pdf, .jpg`), evitando la ejecución de scripts maliciosos o binarios no autorizados en el almacenamiento del NAS.
- **Auditoría mediante Logging Centralizado:** Se desarrolló un sistema de registro de eventos (*logging*) que documenta cada acción realizada en el servidor (inicios de sesión, modificaciones de archivos y errores). Este archivo de auditoría es crucial para el análisis forense y la detección temprana de comportamientos anómalos o «intentos de intrusión». mediante el rastreo de peticiones inusuales.

II-G. Protección contra Vulnerabilidades Web Comunes

En el backend desarrollado con Flask, se integraron protecciones específicas contra ataques dirigidos:

1. **Protección CSRF (Cross-Site Request Forgery):** Es fundamental para evitar que atacantes engañen a un usuario autenticado para realizar acciones no deseadas (como cambiar contraseñas). Se implementaron *tokens* únicos por sesión que validan que cada petición provenga genuinamente de la interfaz del usuario.
2. **Rate Limiting (Limitación de Tasa):** Dado que el hardware cuenta con recursos críticos (Intel Atom), el *Rate Limiting* es vital. Esta medida limita el número de peticiones que una dirección IP puede realizar en un periodo determinado, previniendo tanto ataques de denegación de servicio (DoS) como el agotamiento prematuro de la memoria RAM y el ancho de banda por parte de bots.

II-H. Implementación del Servidor de Videojuegos (Halo CE)

Para diversificar las funciones del hardware, se desplegó un servidor dedicado de *Halo: Custom Edition*. Este proceso requirió la emulación de binarios de Windows sobre Linux mediante una capa de compatibilidad y la gestión de sesiones persistentes.

II-I. Capa de Compatibilidad y Dependencias

Dado que *haloded.exe* es un binario de 32 bits. Se instaló **Wine** en su variante de 32 bits junto con **screen** para permitir que el proceso se ejecute en segundo plano sin una sesión SSH activa:

```
sudo dpkg --add-architecture i386
sudo apt update
sudo apt install wine32 wine-stable screen
```

Listing 6: Preparación del entorno para binarios de 32 bits

II-J. Obtención de Binarios y Configuración del Motor

Los archivos base se obtuvieron del repositorio oficial de la comunidad [4]. La configuración del servidor se centralizó en un archivo de inicialización denominado `init.txt`, donde se definieron los parámetros de red y la visibilidad global:

```
sv_name "Server_Android_PC_Logic"
sv_public 1
sv_maxplayers 16
sv_port 2302
sv_mapcycle_begin
```

Listing 7: Archivo init.txt del servidor de Halo

II-K. Ejecución y Persistencia mediante Screen

Para garantizar que el servidor permanezca activo tras cerrar la consola de comandos, se utilizó un multiplexor de terminales. El comando de ejecución integra Wine para el manejo del binario y vincula el puerto UDP previamente abierto en el firewall:

```
# Ejecuci n en una sesi n aislada
screen -S halo_server wine haloded.exe -port 2302 -c init.txt
```

III. Resultados y Evidencias

El sistema permite la gestión de archivos y la conexión de jugadores externos con una latencia mínima. El uso de *logging* rotativo (mencionado en `app.py`) previene que el disco duro limitado de la netbook se llene con archivos de registro.

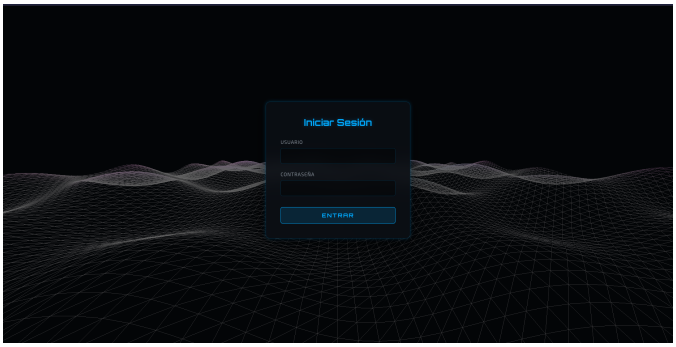

Figura 4: Interfaz de usuario con renderizado WebGL (Three.js) funcionando sobre el hardware optimizado.

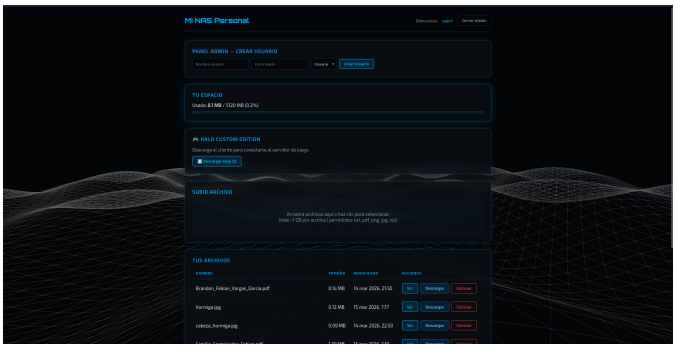

Figura 5: Dashboard de administración y gestión de archivos.

IV. Conclusión

La Acer Aspire One, a pesar de sus 2 GB de RAM, se posiciona como una solución viable para servicios de red ligeros. La clave del éxito radicó en la correcta configuración del firewall, la persistencia de servicios y el desplazamiento de la carga computacional gráfica hacia el cliente. Este proyecto demuestra que el hardware .ºbsoleto.es funcional bajo una arquitectura de software inteligente.

Referencias

[1] Freedesktop.org, "systemd Service Unit Configuration," 2025.
[2] J. Postel, "User Datagram Protocol (UDP)," RFC 768, 1980.
[3] Mr.doob, "Three.js: JavaScript 3D Library," 2026.
[4] Halo Custom Edition Community, "Halo CE Server Files and Documentation," 2026. [En línea]. Disponible en: https://halocustomedition.github.io/Halo/
[5] WineHQ, "Wine Is Not an Emulator (Recursive Acronym)," 2026. [En línea]. Disponible en: https://www.winehq.org/